```
  play_the_song
end
```

The unless statement does support else, but it's always clearer to switch to an if statement in these cases.

Finally, Ruby also supports the ternary operator conditional expression, as also seen in JavaScript and Java:

```
cost = duration > 180 ? 0.35 : 0.25
```

A ternary expression returns the value of the expression either before or after the colon, depending on whether the boolean expression before the question mark is true or false. In the previous example, if the duration is greater than three minutes, the expression returns 0.35. For shorter durations, it returns 0.25. The result is then assigned to cost.

if and unless Modifiers

Statement modifiers let you tack conditional statements onto the end of a normal statement:

```
mon, day, year = $1, $2, $3 if date =~ /(\d\d)-(\d\d)-(\d\d)/
puts "a = #{a}" if $DEBUG
print total unless total.zero?
```

For an if modifier, the preceding expression will be evaluated only if the condition is true. The unless modifier works the other way around:

```
File.foreach("/etc/passwd") do |line|
  next if line =~ /^#/             # Skip comments
  parse(line) unless line =~ /^$/   # Don't parse empty lines
end
```

case Expressions

The Ruby case expression is a powerful beast: a multiway if on steroids.

A Ruby case expression can be written as basically a series of if statements; it lets you list a series of conditions and execute a statement corresponding to the first one that's true:

```
case
when song.name == "Misty"
  puts "Not again!"
when song.duration > 120
  puts "Too long!"
when Time.now.hour > 21
  puts "It's too late"
else
  song.play
end
```

The else clause at the end is optional, and is evaluated if none of the earlier expressions are true.

Note that standard Ruby style has the when statements at the same level as the parent case, not indented.